Session 13 (AIML) - Assignment Questions

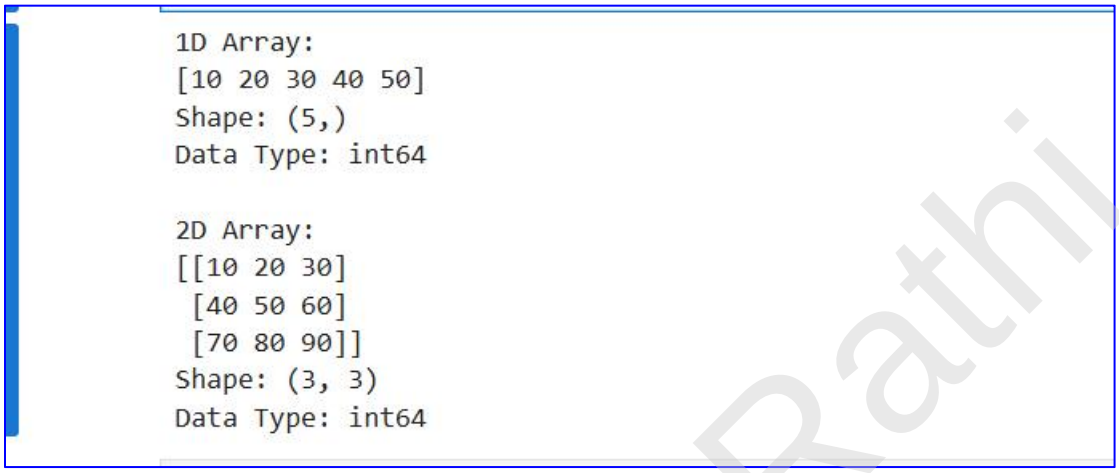
Name**: Vaishnavi Praveen Rathi**
College**: Zeal Polytechnic, Narhe, Pune**
Branch**: Artificial Intelligence and Machine Learning (AIML)**
Domain**: AIML (Artificial Intelligence and Machine Learning)**

Github Repo Link : https://github.com/Vaishnavi-Rathi-123/mountreach.solutions.internship-aiml-tasks_assignments

Github File Link : https://github.com/Vaishnavi-Rathi-123/mountreach.solutions.internship-aiml-tasks_assignments/blob/main/Assignment13.ipynb

Q1.Basic Array Creation

Output Screenshots:



```
1D Array:
[10 20 30 40 50]
Shape: (5,)
Data Type: int64

2D Array:
[[10 20 30]
 [40 50 60]
 [70 80 90]]
Shape: (3, 3)
Data Type: int64
```

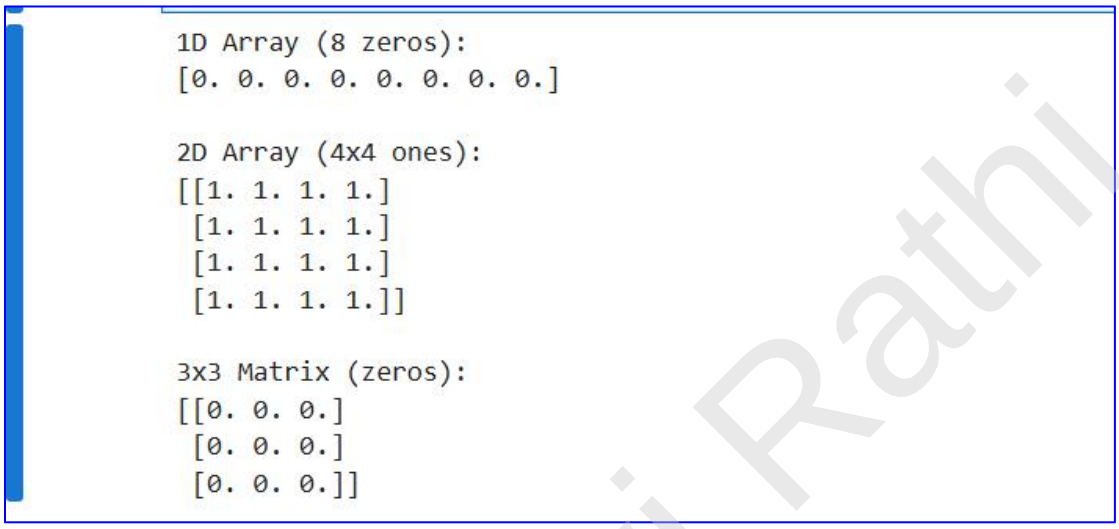
Program Code:

```
""" ASSIGNMENT 13 """

'''
Q1. (Basic Array Creation)

Write a program to:
a) Create a 1D array with elements [10, 20, 30, 40, 50] using np.array().
b) Create a 2D array (3x3) using np.array().
Print both arrays along with their shape and data type (dtype).
'''

import numpy as np

# 1D array creation
arr1 = np.array([10, 20, 30, 40, 50])

# 2D array creation (3x3 matrix)
arr2 = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

# Displaying 1D array details
```

```
print("1D Array:")
print(arr1)
print("Shape:", arr1.shape)
print("Data Type:", arr1.dtype)

# Displaying 2D array details
print("\n2D Array:")
print(arr2)
print("Shape:", arr2.shape)
print("Data Type:", arr2.dtype)
```

Q2. np.zeros() and np.ones()

Output Screenshot:



```
1D Array (8 zeros):
[0. 0. 0. 0. 0. 0. 0. 0.]

2D Array (4x4 ones):
[[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]

3x3 Matrix (zeros):
[[0. 0. 0.]
 [0. 0. 0.]
 [0. 0. 0.]]
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q2. (np.zeros() and np.ones())

Create the following using NumPy:
a) A 1D array of 8 zeros.
b) A 2D array of shape (4, 4) filled with ones.
c) A 3x3 matrix of zeros.
Print all arrays with proper labels.
'''

import numpy as np

# a) 1D array of 8 zeros
arr1 = np.zeros(8)

# b) 2D array (4x4) filled with ones
arr2 = np.ones((4, 4))

# c) 3x3 matrix of zeros
arr3 = np.zeros((3, 3))
```

```
# Print results
print("1D Array (8 zeros):")
print(arr1)

print("\n2D Array (4x4 ones):")
print(arr2)

print("\n3x3 Matrix (zeros):")
print(arr3)
```

Q3.np.arange()

Output Screenshot:

```
a) 0 to 20:
[ 0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19 20]

b) Even numbers (10 to 50):
[10 12 14 16 18 20 22 24 26 28 30 32 34 36 38 40 42 44 46 48 50]

c) 5 to 100 step 5:
[  5  10  15  20  25  30  35  40  45  50  55  60  65  70  75  80  85  90
  95 100]
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q3. (np.arange())

Use np.arange() to create and print the following arrays:
a) Numbers from 0 to 20 (step 1).
b) Even numbers from 10 to 50.
c) Numbers from 5 to 100 with step of 5.
'''

import numpy as np

# a) Numbers from 0 to 20 (step 1)
arr1 = np.arange(0, 21, 1)

# b) Even numbers from 10 to 50
arr2 = np.arange(10, 51, 2)

# c) Numbers from 5 to 100 with step of 5
arr3 = np.arange(5, 101, 5)

# Print results
print("a) 0 to 20:")
print(arr1)
```

```
print("\nb) Even numbers (10 to 50):")
print(arr2)

print("\nc) 5 to 100 step 5:")
print(arr3)
```

Q4. np.linspace()

Output Screenshot:

```
a) 10 values between 0 and 5:
[0.         0.55555556 1.11111111 1.66666667 2.22222222 2.77777778
 3.33333333 3.88888889 4.44444444 5.        ]
Length: 10

b) 15 values between -10 and 10:
[-10.          -8.57142857  -7.14285714  -5.71428571  -4.28571429
  -2.85714286  -1.42857143   0.           1.42857143   2.85714286
   4.28571429   5.71428571   7.14285714   8.57142857  10.        ]
Length: 15
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q4. (np.linspace())

Use np.linspace() to create:
a) 10 equally spaced numbers between 0 and 5.
b) 15 equally spaced numbers between -10 and 10.
Print the arrays and their length.
'''

import numpy as np

# a) 10 equally spaced numbers between 0 and 5
arr1 = np.linspace(0, 5, 10)

# b) 15 equally spaced numbers between -10 and 10
arr2 = np.linspace(-10, 10, 15)

# Print results
print("a) 10 values between 0 and 5:")
print(arr1)
print("Length:", len(arr1))

print("\nb) 15 values between -10 and 10:")
print(arr2)
print("Length:", len(arr2))
```

Q5. Random Arrays

Output Screenshot:

```
a) 1D Random Array (0 to 1):
[0.69665659 0.67484048 0.2018991  0.8593794  0.45606766 0.78925791
 0.29870547 0.27818278 0.1160723  0.32786173]

b) 3x3 Normal Distribution Matrix:
[[ 0.85402343  0.86548821 -0.04488077]
 [ 0.62661225  0.23127582  1.72994386]
 [-2.02808039 -0.3628235  -0.28782973]]

c) 4x5 Random Integer Array (10 to 50):
[[43 45 18 24 28]
 [40 47 17 25 35]
 [14 29 25 39 11]
 [41 11 32 16 11]]
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q5. (Random Arrays)

Write a program to generate:
a) A 1D array of 10 random numbers between 0 and 1 using np.random.rand().
b) A 3x3 matrix of random numbers from standard normal distribution using
   np.random.randn().
c) A 2D array (4x5) of random integers between 10 and 50 using
np.random.randint().
'''

import numpy as np

# a) 1D array (0 to 1)
arr1 = np.random.rand(10)

# b) 3x3 normal distribution matrix
arr2 = np.random.randn(3, 3)

# c) 4x5 random integers between 10 and 50
arr3 = np.random.randint(10, 51, (4, 5))
```

```
# Print results
print("a) 1D Random Array (0 to 1):")
print(arr1)

print("\nb) 3x3 Normal Distribution Matrix:")
print(arr2)

print("\nc) 4x5 Random Integer Array (10 to 50):")
print(arr3)
```

Q6. Vectors and Basic Operations

Output Screenshot:

```
Vector 1: [2 4 6 8]
Vector 2: [1 3 5 7]

Addition:
[ 3  7 11 15]

Subtraction:
[1 1 1 1]

Element-wise Multiplication:
[ 2 12 30 56]

Dot Product:
100
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q6. (Vectors and Basic Operations)

Create two vectors:
v1 = np.array([2, 4, 6, 8])
v2 = np.array([1, 3, 5, 7])

Perform and print:
- Addition
- Subtraction
- Element-wise multiplication
- Dot product
'''

import numpy as np

# Create vectors
v1 = np.array([2, 4, 6, 8])
v2 = np.array([1, 3, 5, 7])

# Operations
```

```
add = v1 + v2
sub = v1 - v2
mul = v1 * v2
dot = np.dot(v1, v2)

# Print results
print("Vector 1:", v1)
print("Vector 2:", v2)

print("\nAddition:")
print(add)

print("\nSubtraction:")
print(sub)

print("\nElement-wise Multiplication:")
print(mul)

print("\nDot Product:")
print(dot)
```

Q7. Matrices and Operations

Output Screenshot:

```
Matrix A:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

Matrix B:
[[9 8 7]
 [6 5 4]
 [3 2 1]]

Matrix Addition (A + B):
[[10 10 10]
 [10 10 10]
 [10 10 10]]

Matrix Multiplication (A @ B):
[[ 30  24  18]
 [ 84  69  54]
 [138 114  90]]

Element-wise Multiplication (A * B):
[[ 9 16 21]
 [24 25 24]
 [21 16  9]]
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q7. (Matrices and Operations)

Create two 3x3 matrices:
A = np.array([[1,2,3],[4,5,6],[7,8,9]])
B = np.array([[9,8,7],[6,5,4],[3,2,1]])

Perform the following:
- Matrix addition
- Matrix multiplication (@ or np.dot())
- Element-wise multiplication (*)
'''
```

```
import numpy as np

# Create matrices
A = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9]])

B = np.array([[9, 8, 7],
              [6, 5, 4],
              [3, 2, 1]])

# Operations
matrix_add = A + B
matrix_mul = A @ B
element_mul = A * B

# Print results
print("Matrix A:")
print(A)

print("\nMatrix B:")
print(B)

print("\nMatrix Addition (A + B):")
print(matrix_add)

print("\nMatrix Multiplication (A @ B):")
print(matrix_mul)

print("\nElement-wise Multiplication (A * B):")
print(element_mul)
```

Q8.Properties of Arrays

Output Screenshot:

```
Matrix:
[[67 11  9 90]
 [72 38 18 69]
 [33 70  9 34]
 [29 60 54 69]]

Shape: (4, 4)
Dimensions (ndim): 2
Size (total elements): 16
Data Type: int32

Minimum Value: 9
Maximum Value: 90
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q8. (Properties of Arrays)

Create a 4x4 matrix of random integers between 1 and 100.
Print:
- Shape
- Dimension (ndim)
- Total number of elements (size)
- Data type
- Minimum value
- Maximum value
'''

import numpy as np

# Create 4x4 random integer matrix (1 to 100)
arr = np.random.randint(1, 101, (4, 4))

# Print array
print("Matrix:")
```

```
print(arr)

# Properties
print("\nShape:", arr.shape)
print("Dimensions (ndim):", arr.ndim)
print("Size (total elements):", arr.size)
print("Data Type:", arr.dtype)

# Min and Max values
print("\nMinimum Value:", arr.min())
print("Maximum Value:", arr.max())
```

Q9.Combined - Random + Reshape + Statistics

Output Screenshot:

```
Original 1D Array:
[19  8  3  2 10  3 36 41 42 50 20 21 29 24  7 40  1 16 21 26]

4x5 Matrix:
[[19  8  3  2 10]
 [ 3 36 41 42 50]
 [20 21 29 24  7]
 [40  1 16 21 26]]

Sum of Matrix: 419
Mean of Matrix: 20.95
Standard Deviation: 14.647440049373813

Maximum value in each row:
[19 50 29 40]
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q9. (Combined - Random + Reshape + Statistics)

Generate a 1D array of 20 random integers between 1 and 50 using
np.random.randint().
Reshape it into a 4x5 matrix.

Print:
- Sum
- Mean
- Standard Deviation
- Maximum value in each row
'''

import numpy as np

# 1D array of 20 random integers (1 to 50)
arr = np.random.randint(1, 51, 20)

# Reshape into 4x5 matrix
matrix = arr.reshape(4, 5)
```

```
# Calculations
total_sum = np.sum(matrix)
mean_val = np.mean(matrix)
std_val = np.std(matrix)
row_max = np.max(matrix, axis=1)

# Print results
print("Original 1D Array:")
print(arr)

print("\n4x5 Matrix:")
print(matrix)

print("\nSum of Matrix:", total_sum)
print("Mean of Matrix:", mean_val)
print("Standard Deviation:", std_val)

print("\nMaximum value in each row:")
print(row_max)
```

Q10. Mini Project - NumPy Application

Output Screenshot:

```
Enter number of elements to generate:  12

Generated Array:
[91 90 31 24 65 56 21 13 41 17 85 31]

Mean: 47.083333333333336
Median: 36.0
Standard Deviation: 28.051911204447766
Minimum: 13
Maximum: 91

Reshaping Array:

Reshaped Matrix:
[[91 90 31 24]
 [65 56 21 13]
 [41 17 85 31]]

Row-wise Sum:
[236 155 174]
```

Program Code:

```
""" ASSIGNMENT 13 """

'''
Q10. (Mini Project - NumPy Application)

Create a Simple Statistics Calculator using NumPy:

Program should:
- Ask user how many numbers to generate
- Generate random numbers (10 to 100)
- Store in 1D array
- Print:
  - Array
  - Mean, Median, Standard Deviation
  - Minimum and Maximum
- Reshape into 2D array (if possible)
- Print row-wise sum
'''
```

```
import numpy as np

try:
    # User input
    n = int(input("Enter number of elements to generate: "))

    if n <= 0:
        raise ValueError("Number must be greater than 0")

    # Generate random numbers
    arr = np.random.randint(10, 101, n)

    # Statistics
    mean_val = np.mean(arr)
    median_val = np.median(arr)
    std_val = np.std(arr)
    min_val = np.min(arr)
    max_val = np.max(arr)

    # Print array and stats
    print("\nGenerated Array:")
    print(arr)

    print("\nMean:", mean_val)
    print("Median:", median_val)
    print("Standard Deviation:", std_val)
    print("Minimum:", min_val)
    print("Maximum:", max_val)

    # Reshape logic
    print("\nReshaping Array:")

    # Find possible shape (simple approach: square-like or 2D fallback)
    rows = int(np.sqrt(n))
    cols = n // rows

    if rows * cols == n:
        reshaped = arr.reshape(rows, cols)
        print("\nReshaped Matrix:")
        print(reshaped)

        print("\nRow-wise Sum:")
        print(np.sum(reshaped, axis=1))
```

```
    else:
        print("Reshape not possible into perfect 2D matrix")

except ValueError as ve:
    print("Input Error:", ve)

except Exception as e:
    print("Unexpected Error:", e)
```
